

APPLIED CRYPTOGRAPHY ASSIGNMENT

CYBER SECURITY
BATCH - II

S.NO	TABLE OF CONTENT	PAGE NO.
1	ASSIGNMENT OVERVIEW	3
2	IMPLEMENTING CHAP ALGORITHM	4-5
3	AES BLOCK CIPHER MODES OF OPERATION	6-7
4	IMPLEMENTING RSA KEY GENERATION ALGORITHM	8-10
5	ENCRYPTING AND DECRYPTING YOUR GROUP NAME USING RSA	10
6	CONCLUSION	11-12
7	REFERENCES	13

OVERVIEW

This assignment aims at practical application and study of basic cryptographic processes in the context of the state of the art cyber security systems. The primary goal is to learn the functionality of authentication, symmetric encryption and asymmetric encryption by applying them in the Python programming language.

The task is broken down into several tasks. First of all, Challenge-Handshake Authentication Protocol (CHAP) is applied to show the secure authentication based on the challenge-response scheme without any password disclosures. Then we will implement and compare two modes of Advanced Encryption Standard (AES) block cipher namely ECB and CBC to obtain a picture of their differences in functioning and the security concerns. Subsequently the assignment discusses RSA key generation, the large prime numbers are generated and used to create the public and private keys based on mathematical concepts including the totient function and modular inverses. Lastly, the group name is encrypted and decrypted safely using RSA encryption and decryption.

On the whole, the assignment enhances theoretical understanding and practical skills in the area of cryptography, which is the way to ensure confidentiality, integrity and authentication of data in real-world systems by using cryptographic algorithms.

The name of the group applied in this assignment is CIPHERCREW.

QUESTION 1 - IMPLEMENTING CHAP ALGORITHM

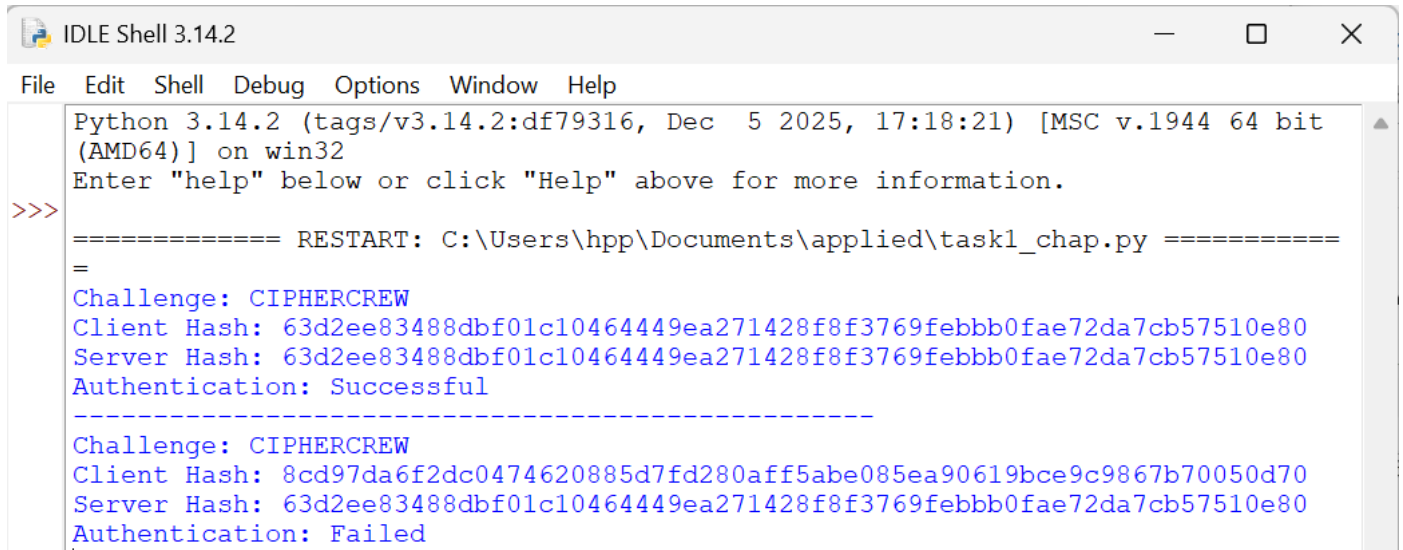
A) Explanation of the Background -

The Challenge-Handshake Authentication Protocol (CHAP) is a tool of authentication to validate a user or system without transmitting the correct password across the network. It is primarily aimed at ensuring that credentials are not intercepted during communication. CHAP operates based on a challenge-response protocol: the server transmits a randomly chosen challenge string, the client mixes that string with his password, hashes the output, and transmits the hash. The server also makes the same hash and compares the hashes.

This will eliminate exposure of passwords since the password will never get sent directly. An attacker will only view a hash value rather than the password even when he or she intercepts the communication. Attackers are not able to reuse previously obtained responses as each challenge is unique, which defends against replay attacks.

The role in CHAP is played by hashing. Hash functions such as MD5 or SHA-256 also transform the combination of challenge and password to a fixed-size digest, which with an input of any password is computationally infeasible to reverse to find the original password. SHA-256 is widely used nowadays, as it ensures tremendous resistance to collision and brute force attack. CHAP also uses the subsequent hashes that make sure that verification is secure without the sensitive credentials being disclosed.

B) Implementation of Challenge-Handshake Authentication Protocol



```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\hpp\Documents\applied\task1_chap.py =====
=
Challenge: CIPHERCREW
Client Hash: 63d2ee83488dbf01c10464449ea271428f8f3769febbb0fae72da7cb57510e80
Server Hash: 63d2ee83488dbf01c10464449ea271428f8f3769febbb0fae72da7cb57510e80
Authentication: Successful
-----
Challenge: CIPHERCREW
Client Hash: 8cd97da6f2dc0474620885d7fd280aff5abe085ea90619bce9c9867b70050d70
Server Hash: 63d2ee83488dbf01c10464449ea271428f8f3769febbb0fae72da7cb57510e80
Authentication: Failed
```

FIGURE 1: OUTPUT OF CHAP ALGORITHM

Explanation of the software developed - The software created has been done in Python to simulate the Challenge-Handshake Authentication Protocol (CHAP). Clientauth() function computes a hash of the challenge and password using the SHA-256 hash. The serververify() service recalculates the hash and compares it with a response of the client. The main() procedure illustrates two instances: the correct password and wrong password with the message of authentication: Successful, and failed respectively. Such an implementation illustrates a safe challenge-response authentication where the real password is not sent.

QUESTION 2 - AES BLOCK CIPHER MODES OF OPERATION

A) Explanation of the Background -

Block ciphers like the AES work in pre-defined block sized data (128 bits). Nevertheless, messages in the real world are normally longer than one block and they may not be an accurate multiple of the block size. Such modes of operation are thus needed to be able to safely encrypt data of arbitrary length and define the way in which successive blocks are operated upon. The absence of modes of operation would cause the encryption of every block separately to leak patterns.

The mode of Electronic Codebook (ECB) encrypts each block of plaintext with the same key. The greatest strength it has is that it is simple and efficient and the blocks could be encrypted and decrypted simultaneously. Nonetheless, ECB is not secure, as the same plaintext blocks are encrypted to give the same ciphertext blocks, which disclose trends in the data. Due to this fact, sensitive information should not be written with ECB.

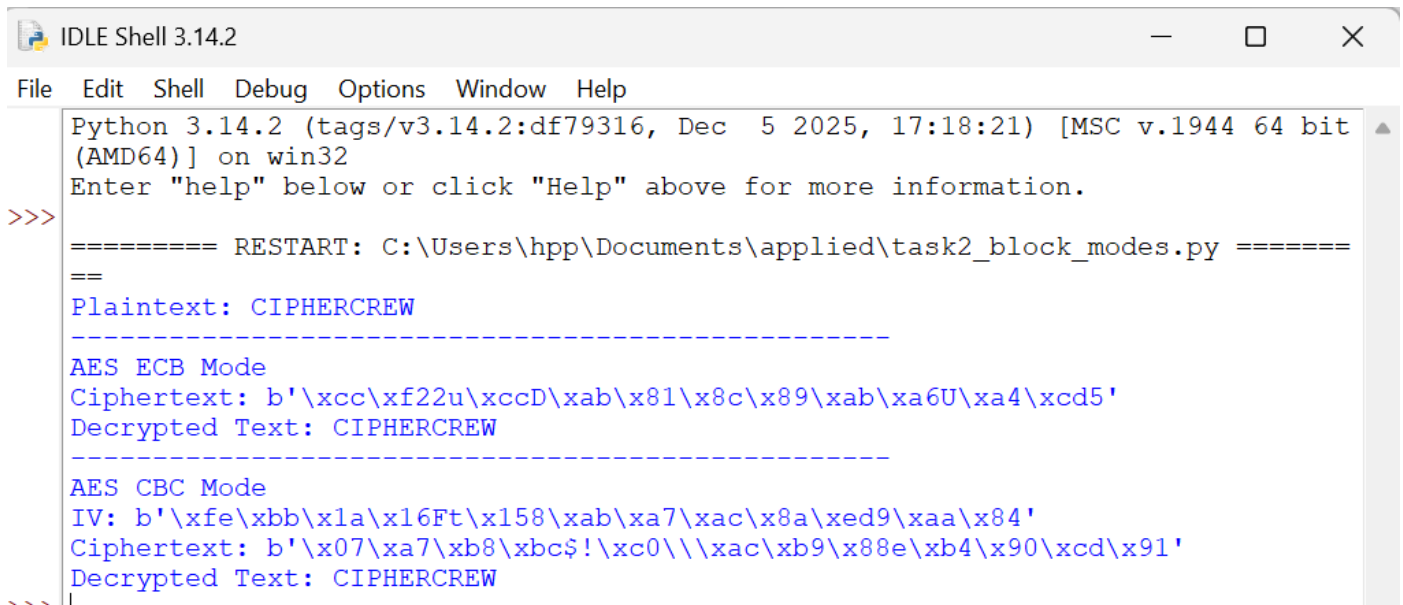
Cipher Block Chaining (CBC) mode enhances better security because it encryption block-by-block uses XOR of the plaintext block and the previous encrypted block. The first block is initialized with a vector (IV). This avoids leakage of patterns and corrects the major weakness of ECB. Nonetheless, CBC encryption cannot be paralleled and has to be handled with padding.

The Cipher Feedback (CFB) mode transforms a block cipher to a stream cipher. It encrypts the former ciphertext (or IV) and XORs with plaintext. CFB has been shown to provide self-synchronization, that is, it can tolerate limited errors in transmissions. It has the downside that it is not parallel and faster than ECB and CTR.

Counter (CTR) mode also transforms the block cipher into a stream cipher by encrypting a counter-value and XOR it with the plaintext. CTR is high speed, efficient and can be completely parallelized. It is however important that the counter value should not be reused using the same key because it would carry security.

To conclude, mode of operation is vital in the secure use of block ciphers and modes have varying security and performance trade-offs.

B) Implementation of AES Block Cipher Modes of Operation



```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\hpp\Documents\applied\task2_block_modes.py =====
Plaintext: CIPHERCREW
-----
AES ECB Mode
Ciphertext: b'\xcc\x22u\xccD\xab\x81\x8c\x89\xab\xa6U\xa4\xcd5'
Decrypted Text: CIPHERCREW
-----
AES CBC Mode
IV: b'\xfe\xbb\x1a\x16Ft\x158\xab\xa7\xac\x8a\xed9\xaa\x84'
Ciphertext: b'\x07\xa7\xb8\xbc$!\xc0\\\xac\xb9\x88e\xb4\x90\xcd\x91'
Decrypted Text: CIPHERCREW
>>>
```

FIGURE 2: OUTPUT OF AES BLOCK CIPHER MODES OF OPERATION

Explanation of the software developed – This program demonstrates the implementation of the AES block cipher using two modes of operation: ECB and CBC. A 16-byte secret key is randomly generated, and the group name is used as the plaintext message. Before encryption, the plaintext is padded to match the AES block size, and after decryption the padding is removed. In ECB mode, each block is encrypted independently, while in CBC mode an initialization vector (IV) is generated to chain blocks securely. The program encrypts and decrypts the plaintext using both modes and prints the ciphertext and decrypted text to verify correctness.

QUESTION 3 - IMPLEMENTING RSA KEY GENERATION ALGORITHM

A) Explanation of the Background -

RSA is a popular cryptographic algorithm of the public key in ensuring security of communication and signatures. It relies on the computational complexity of the factoring of large composite numbers which render computationally infeasible to an attacker to extract the private key out of the public key. RSA takes two keys, a public key, which is employed in either encryption or verification, and a private key, which is applied in the decryption or signing.

RSA key generation starts with the choice of two large random prime numbers, normally hundreds of bits in length. These primes are multiplied to give the RSA modulus $N = p \times q$ that is contained in the public and the private keys. Security of RSA depends on the size of N since it is very hard to factor it back to p and q when the N is big.

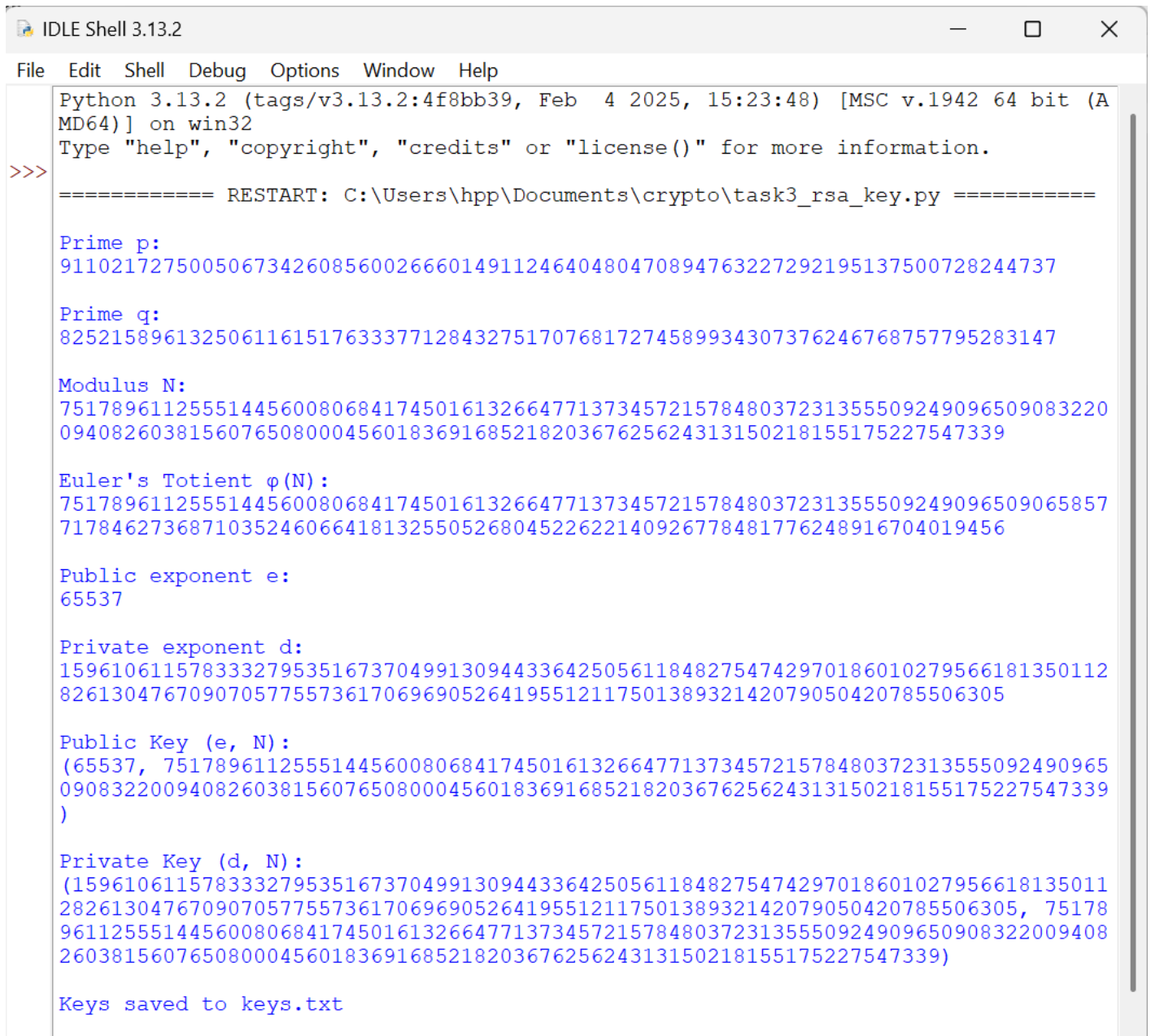
Then, Euler Totient Function $\phi(N) = (p-1)(q-1)$ is computed. This value is the COUNT of numbers smaller than N which are coprime numbers to it. An exponent e is then picked, the exponent being 65537, since it provides good security and can be easily computed.

The private exponent d is found by computing the modular inverse of e with respect to $\phi(N)$ that is: $d \equiv e^{-1} \pmod{\phi(N)}$.

This guarantees that encryption and decryption are connected yet computationally infeasible to invert without having knowledge of the secret code.

Once (e, N) and (d, N) have been calculated, the program writes the values and saves them in a keys.txt file which will be used later in the program. This file incorporates the elements of key (public and private) which would be needed in encryption and decryption operations.

B) Implementation of RSA Key Generation Algorithm



```
IDLE Shell 3.13.2
File Edit Shell Debug Options Window Help
Python 3.13.2 (tags/v3.13.2:4f8bb39, Feb  4 2025, 15:23:48) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\hnp\Documents\crypto\task3_rsa_key.py =====

Prime p:
91102172750050673426085600266601491124640480470894763227292195137500728244737

Prime q:
82521589613250611615176333771284327517076817274589934307376246768757795283147

Modulus N:
751789611255514456008068417450161326647713734572157848037231355509249096509083220
0940826038156076508000456018369168521820367625624313150218155175227547339

Euler's Totient  $\phi(N)$ :
751789611255514456008068417450161326647713734572157848037231355509249096509065857
7178462736871035246066418132550526804522622140926778481776248916704019456

Public exponent e:
65537

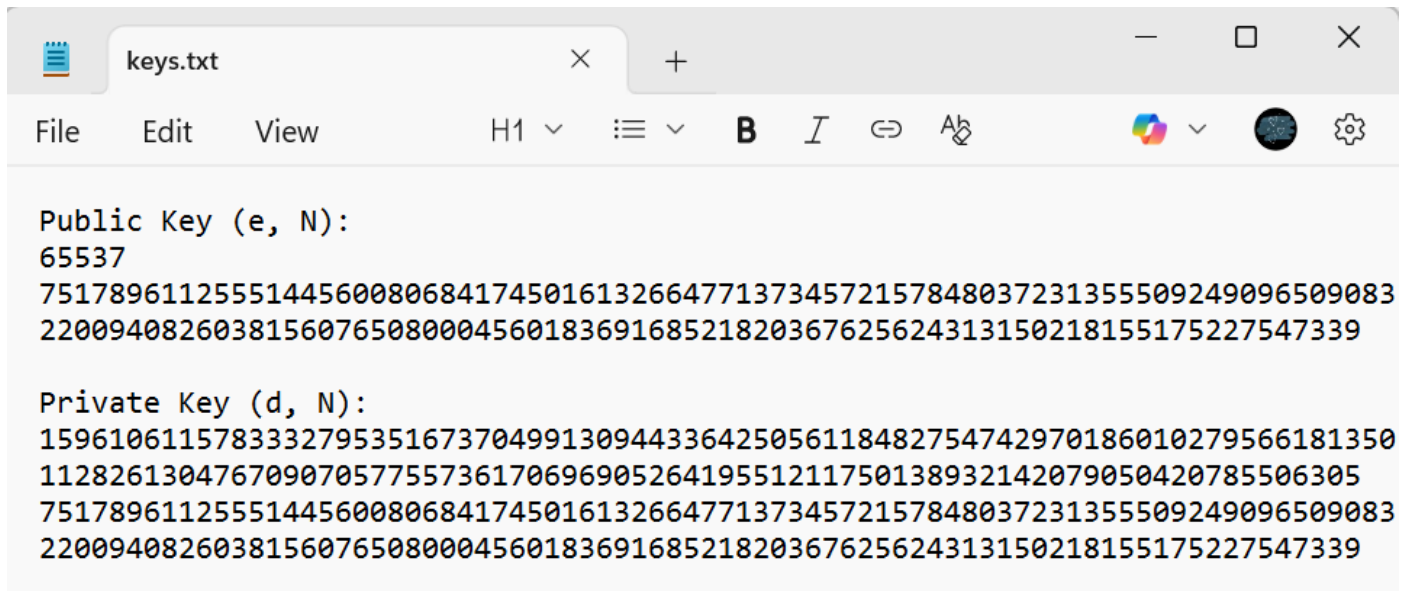
Private exponent d:
159610611578333279535167370499130944336425056118482754742970186010279566181350112
8261304767090705775573617069690526419551211750138932142079050420785506305

Public Key (e, N):
(65537, 7517896112555144560080684174501613266477137345721578480372313555092490965
090832200940826038156076508000456018369168521820367625624313150218155175227547339
)

Private Key (d, N):
(15961061157833327953516737049913094433642505611848275474297018601027956618135011
28261304767090705775573617069690526419551211750138932142079050420785506305, 75178
961125551445600806841745016132664771373457215784803723135550924909650908322009408
26038156076508000456018369168521820367625624313150218155175227547339)

Keys saved to keys.txt
```

FIGURE 3: OUTPUT OF RSA KEY GENERATION ALGORITHM



```
Public Key (e, N):
65537
751789611255514456008068417450161326647713734572157848037231355509249096509083
2200940826038156076508000456018369168521820367625624313150218155175227547339

Private Key (d, N):
159610611578333279535167370499130944336425056118482754742970186010279566181350
1128261304767090705775573617069690526419551211750138932142079050420785506305
751789611255514456008068417450161326647713734572157848037231355509249096509083
2200940826038156076508000456018369168521820367625624313150218155175227547339
```

FIGURE 4: OUTPUT OF RSA KEY GENERATION ALGORITHM - KEYS FILE

Explanation of the software developed – The software developed does not use external libraries when generating RSA keys. It starts with the generation of two 256-bits prime numbers by a custom MillerRabin probabilistic primality test. The RSA modulus N is composed of these primes which are multiplied. Euler's Totient $\phi(N) = (p-1)(q-1)$. An appropriate standard public exponent $e = 65537$ is used in order to enable safe and effective encryption. Intended computation is the computation of the private exponent d with the help of the extended Euclidean algorithm, where the modular inverse of e is found modulo $\phi(N)$. The program generates all essential elements and puts both the public and the private key pairs in a keys.txt file.

QUESTION 4: ENCRYPTING AND DECRYPTING YOUR GROUP NAME USING RSA

A) Explanation of the Background -

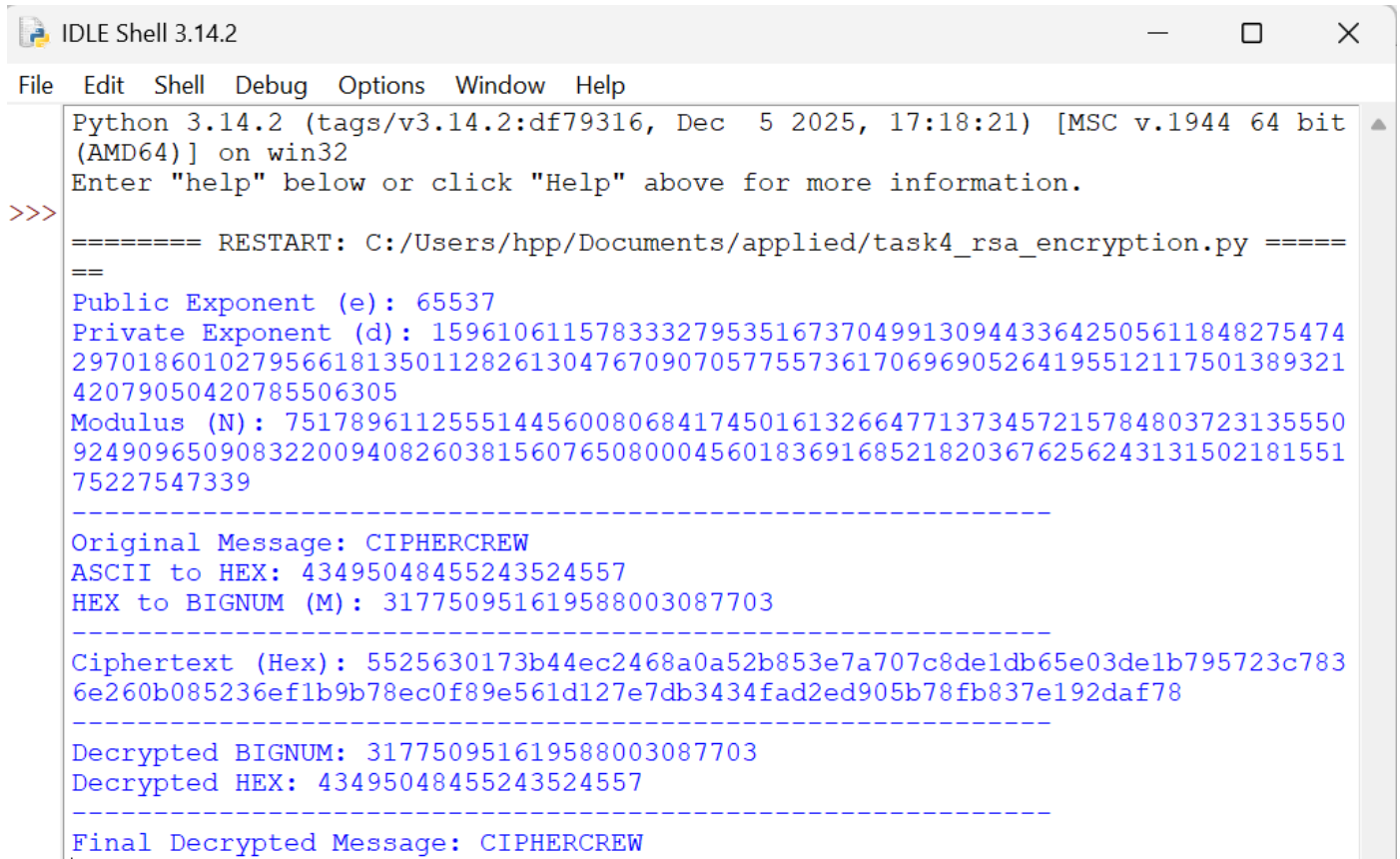
RSA is a popular type of the key cryptography algorithms and it is grounded on the principle of the asymmetric encryption. RSA is also different as opposed to symmetric cryptography in the sense that there are two keys, which are mathematically related i.e. a public key and a private key. The encryption is executed using the public key in which it can be openly shared and the decryption is done using the private key which is not openly available. The encryption of RSA is also dependent on the computational complexity of multiplying a very large number into the two prime numbers of the number.

RSA algorithm starts with the process of generating two large prime numbers, which are normally hundreds of bits long. These primes are multiplied to get the modulus N that is a fundamental element of both the private and the public keys. The totient Function $\phi(N)$ by Euler is then calculated and used in computing the private exponent. A public exponent e is selected and the private exponent d is computed in a manner that $\phi(N)$ is the modular inverse of d . The key is public (e, N) and the key is privately known (d, N) .

Prior to encryption, a plaintext message has to be coded into a mathematical format in numbers. It is generally accomplished by encoding the ASCII characters of the message into a hexadecimal form and turn into a large integer, referred to as a BIGNUM. The RSA encryption is being conducted following the formula $C = M^e \bmod N$ where C is the ciphertext, and M is the message. The process of decryption involves the use of the private key and the formula $M = C^d \bmod N$ which will help retrieve the originally encrypted message.

In the contemporary communication systems, RSA is often applied in secure exchange of keys, digital signature and security of sensitive information. Its advantage is that it has the modular exponentiation and is hard to reverse without the private key.

B) Implementation of Encrypting and Decrypting Your Group Name using RSA

The image shows a screenshot of the IDLE Shell 3.14.2 window. The title bar reads 'IDLE Shell 3.14.2'. The menu bar includes 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The shell prompt is 'Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32'. Below the prompt, it says 'Enter "help" below or click "Help" above for more information.' The user has entered '>>>'. The output shows the execution of a Python script 'task4_rsa_encryption.py'. The script displays the public and private exponents, the modulus, the original message 'CIPHERCREW', its conversion to hex and then to a bignum, the resulting ciphertext in hex, and finally the decrypted bignum and hex, which are converted back to the original message 'CIPHERCREW'.

```
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/hpp/Documents/applied/task4_rsa_encryption.py =====
Public Exponent (e): 65537
Private Exponent (d): 15961061157833327953516737049913094433642505611848275474
297018601027956618135011282613047670907057755736170696905264195512117501389321
42079050420785506305
Modulus (N): 75178961125551445600806841745016132664771373457215784803723135550
924909650908322009408260381560765080004560183691685218203676256243131502181551
75227547339
-----
Original Message: CIPHERCREW
ASCII to HEX: 43495048455243524557
HEX to BIGNUM (M): 317750951619588003087703
-----
Ciphertext (Hex): 5525630173b44ec2468a0a52b853e7a707c8de1db65e03de1b795723c783
6e260b085236ef1b9b78ec0f89e561d127e7db343fad2ed905b78fb837e192daf78
-----
Decrypted BIGNUM: 317750951619588003087703
Decrypted HEX: 43495048455243524557
-----
Final Decrypted Message: CIPHERCREW
```

FIGURE 5: OUTPUT OF ENCRYPTION AND DECRYPTION USING RSA

Explanation of the software developed – The Python software created illustrates the RSA encryption and decryption of a plaintext message represented by a group name. The program loads the RSA public and private keys using a file known as keys.txt. The initial message undergoes conversion of ASCII to hexadecimal and further to a BIGNUM to be mathematically manipulated. RSA encryption is achieved with the help of the public exponent whereas the decryption is performed with the help of the private exponent. The value of the decrypted hexadecimal is translated back to ASCII and confirm that it is accurate. The output proves that the result message is similar to the first input, which is a confirmation of the RSA algorithm.

REFERENCES

- Stallings, W., 2011. Cryptography And Networksecurity Principles Andpractice.
- Kessler, G.C., 2003. An overview of cryptography.
- Diffie, W. and Hellman, M.E., 2022. New directions in cryptography. In Democratizing cryptography: the work of Whitfield Diffie and Martin Hellman (pp. 365-390).
- Menezes, A.J., Van Oorschot, P.C. and Vanstone, S.A., 2018. Handbook of applied cryptography. CRC press.
- Dakhni, R., Sandilkar, S. and Balaram, D., 2023. Implementation of rsa algorithm for encryption and decryption recent technology trends in computer technology. vol, 3, pp.2581-9429.
- Dworkin, M., 2001. Recommendation for block cipher modes of operation. NIST special publication, 800, p.38B.
- Dworkin, M., 2001. Recommendation for block cipher modes of operation. NIST special publication, 800, p.38B.